Programming Languages and Computation

Week 7: Operational Semantics

1 Operational Semantics

This section is about the small-step operational semantics of While programs as given by the relation $\rightarrow \subseteq \mathcal{C} \times \mathcal{C}$ where the configurations are either a statement and a state or a state $\mathcal{C} = (\mathsf{S} \times \mathsf{State}) \cup \mathsf{State}$, which is defined by these following rules:

$$\frac{}{\langle \mathsf{skip}, \sigma \rangle \rightarrow \sigma} \qquad \frac{}{\langle x \leftarrow e, \sigma \rangle \rightarrow \sigma[x \mapsto [\![e]\!]_{\mathcal{A}}(\sigma)]}$$

$$\frac{\langle S_1, \sigma_1 \rangle \rightarrow \sigma_2}{\langle S_1; S_2, \sigma_1 \rangle \rightarrow \langle S_2, \sigma_2 \rangle} \qquad \frac{\langle S_1, \sigma_1 \rangle \rightarrow \langle S_1', \sigma_2 \rangle}{\langle S_1; S_2, \sigma_1 \rangle \rightarrow \langle S_1'; S_2, \sigma_2 \rangle}$$

$$\frac{}{\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma \rangle \rightarrow \langle S_1, \sigma \rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \top$$

$$\frac{}{\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma \rangle \rightarrow \langle S_2, \sigma \rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \bot$$

$$\frac{}{\langle \mathsf{while}\ e\ \{S\}, \sigma \rangle \rightarrow \sigma}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \bot$$

$$\frac{}{\langle \mathsf{while}\ e\ \{S\}, \sigma \rangle \rightarrow \langle S; \mathsf{while}\ e\ \{S\}, \sigma \rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \top$$

Figure 1: Operational semantics for While.

* 1. Calculate the terminal state and write down a trace for the statement $x \leftarrow 1; x \leftarrow 2; x \leftarrow 3$ with the initial state $[\,]$ using the rules in Figure 1. Remember, variables are assigned 0 by default.

* 2. Calculate the terminal state and write down a trace for the statement $x \leftarrow 1; x \leftarrow x * 2; x \leftarrow x + y$ with the initial state $[x \mapsto 2, y \mapsto 2]$ using the rules in Figure 1.

* 3. Find a state σ such that $\langle x \leftarrow 1; y \leftarrow x * 2, [\,] \rangle \rightarrow^* \sigma$. You should provide the corresponding trace.

* 4. Compute the final state for the program if $x \leq y$ $\{x \leftarrow y\}$ else $\{y \leftarrow x\}$ when executed in each of the following states:

- $[]$
- $[x \mapsto 2,\ y \mapsto 3]$
- $[x \mapsto 4,\ y \mapsto 2]$

* 5. Find a state $\sigma \in \mathsf{State}$ such that $\mathsf{while}\ !(x \leq -1)\ \{x \leftarrow x + d\}, [d \mapsto -1] \rightarrow^* \sigma$. You should provide the corresponding trace.

* 6. Find a state $\sigma \in \mathsf{State}$ such that $\langle x \leftarrow 2; y \leftarrow x * y, \sigma\rangle \rightarrow^* [x \mapsto 2,\ y \mapsto 4]$. You should provide the corresponding trace.

** 7. Suppose $e \in \mathcal{B}$ is a Boolean expression that is semantically equivalent to false. Prove that $\langle \mathsf{while}\ e\ \{S\}, \sigma\rangle \rightarrow \sigma$ for any state $\sigma \in \mathsf{State}$.

** 8. Suppose $S_1, S_2 \in \mathcal{S}$ are two statements such that $\langle S_1, \sigma\rangle \rightarrow^* \sigma'$ and $\langle S_2, \sigma\rangle \rightarrow^* \sigma'$ for some states $\sigma, \sigma' \in \mathsf{State}$. Prove that $\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma\rangle \rightarrow^* \sigma'$ for any Boolean expression $e \in \mathcal{B}$.

** 9. Prove that, for any statement $S \in \mathcal{S}$ and any Boolean expression $e \in \mathcal{B}$, the statement $\mathsf{if}\ e\ \{S\}\ \mathsf{else}\ \{S\}$ is semantically equivalent to S.

** 10. Prove that, for any statements $S_1, S_2 \in \mathcal{S}$ and any Boolean expression $e \in \mathcal{B}$, the statement $\mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}$ is semantically equivalent to $\mathsf{if}\ !e\ \{S_2\}\ \mathsf{else}\ \{S_1\}$.

** 11. Suppose we introduce a new language construct "do S while e" where $S \in \mathcal{S}$ is a statement and $e \in \mathcal{B}$ is a Boolean expression. The operational semantics for this construct is given by the following inference rules:

$$\frac{}{\langle \mathsf{do}\ \{S\}\ \mathsf{while}\ e, \sigma\rangle \rightarrow \langle S;\ \mathsf{if}\ e\ \{\mathsf{do}\ \{S\}\ \mathsf{while}\ e\}\ \mathsf{else}\ \{\mathsf{skip}\}, \sigma\rangle}$$

(a) Find a state $\sigma \in \mathsf{State}$ such that $\langle \mathsf{do}\ \{x \leftarrow x + 1\}\ \mathsf{while}\ x \leq 1, []\rangle \rightarrow^* \sigma$ and give the associated trace.

(b) For a given statement $S \in \mathcal{S}$ and a Boolean expression $e \in \mathcal{B}$ find a While program that is equivalent to the program $\mathsf{do}\ \{S\}\ \mathsf{while}\ e$ but does not use the new construct. That is, find a statement $S' \in \mathcal{S}$ such that:

$$\langle S', \sigma\rangle \rightarrow^* \sigma' \Leftrightarrow \langle \mathsf{do}\ \{S\}\ \mathsf{while}\ e, \sigma\rangle \rightarrow^* \sigma'$$

You do not need to prove that your answer is correct but should provide a trace for $\langle S', []\rangle \rightarrow^* \sigma$ where S is given to be the statement $x \leftarrow x + 1$, where e is given to be the expression $x \leq 1$, and where σ is the state from part (a).

** 12. Suppose we introduce a new language construct for x do S where $S \in \mathcal{S}$ is a statement and $x \in \mathsf{Var}$ is a variable The operational semantics for this construct is given by the following inference rules:

$$\frac{}{\langle \text{for } x \text{ do } S, \sigma\rangle \to \sigma}\ \sigma(x) \leq 0 \qquad \frac{}{\langle \text{for } x \text{ do } S, \sigma_1\rangle \to \langle S;\ \text{for } x \text{ do } S, \sigma[x \mapsto \sigma(x)-1]\rangle}\ \sigma(x) > 0$$

(a) Find a state $\sigma \in \mathsf{State}$ such that $\langle \text{for } x \text{ do } y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 3]\rangle \to^* \sigma$ and give the associated trace.

(b) For a given statement $S \in \mathcal{S}$ and a variable $x \in \mathsf{Var}$ find a While program that is equivalent to the program for x do S but does not use the new construct. That is, find a statement $S' \in \mathcal{S}$ such that:

$$\langle S', \sigma\rangle \to^* \sigma' \iff \langle \text{for } x \text{ do } S, \sigma\rangle \to^* \sigma'$$

You do not need to prove that your answer is correct but should provide a trace for $\langle S', [x \mapsto 3]\rangle \to^* \sigma$ where S is given to be the statement $y \leftarrow y + x;\ x \leftarrow x - 2$ and σ is the state from part (a).

*** 13. Show that, if $y \notin \mathsf{FV}(e_1)$ and $x \notin \mathsf{FV}(e_2)$ where x and y are distinct variables, then the statements $x \leftarrow e_1;\ y \leftarrow e_2$ and $y \leftarrow e_2;\ x \leftarrow e_1$ equivalent in the sense that $\langle x \leftarrow e_1;\ y \leftarrow e_2, \sigma\rangle \to^* \sigma'$ if and only if $\langle y \leftarrow e_2;\ x \leftarrow e_1, \sigma\rangle \to^* \sigma'$. You may use results from the previous worksheet.

*** 14. The set of variables *modified* by a statement is defined by recursion as follows:

$$\begin{aligned}
&\mathsf{mod}(\mathsf{skip}) = \emptyset \\
&\mathsf{mod}(x \leftarrow e) = \{x\} \\
&\mathsf{mod}(S_1;\ S_2) = \mathsf{mod}(S_1) \cup \mathsf{mod}(S_2) \\
&\mathsf{mod}(\text{if } e \text{ then } S_1 \text{ else } S_2) = \mathsf{mod}(S_1) \cup \mathsf{mod}(S_2) \\
&\mathsf{mod}(\text{while } e \text{ do } S) = \mathsf{mod}(S)
\end{aligned}$$

(a) Prove that if $\langle S_1, \sigma_1\rangle \to \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$.

(b) Prove that if $\langle S_1, \sigma_1\rangle \to \langle S_2, \sigma_2\rangle$ then $\mathsf{mod}(S_2) \subseteq \mathsf{mod}(S_1)$ and $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. You may use the previous result.

(c) Prove that if $\langle S_1, \sigma_1\rangle \to^* \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. You may use the previous results, and the fact that $\gamma_1 \to^* \gamma_2$ if, and only if, $\gamma_1 \to^n \gamma_2$ for some $n \geq 0$.